



**National University of Computer and Emerging
Sciences**

Islamabad Campus

Fundamentals of Software Engineering

**UniGateway: AI-Powered University
Admission and Preparation Hub**

Final Project Report (Iteration 3)

Project Team:

M-Faisal (24I-6077)

Talha Habib (24I-0029)

Abdul Ghafoor (24I-0118)

Department of Artificial Intelligence

May 3, 2026

Contents

1	Introduction	5
1.1	Project Background	5
1.2	Problem Statement	5
1.3	Objectives	5
1.4	Scope of the Project	6
1.5	Overview of the System	6
2	System Analysis and Requirements	7
2.1	Existing System	7
2.2	Proposed System	7
2.3	Stakeholders Identification	7
2.4	Functional Requirements	7
2.5	Non-Functional Requirements	8
2.6	Feasibility Analysis	9
3	System Design (UML Diagrams)	10
3.1	System Architecture	10
3.2	Use Case Diagram & Descriptions	11
3.3	Activity Diagrams	16
3.4	Sequence Diagrams	20
3.5	Class Diagram	23
3.6	Database Design (ER Diagram / Schema)	23
3.7	Interface Design (Wireframes / Prototypes)	25
4	Implementation	26
4.1	Development Environment	26
4.2	System Modules Description	26
4.3	Frontend Implementation	27
4.4	Backend Implementation	27
4.5	Database Implementation	27
4.6	Integration of System Components	27
4.7	Key Functionalities (with Screenshots)	27
5	Testing and Evaluation	33
5.1	Testing Strategy	33
5.2	Test Cases and Results	33
5.3	Boundary Value Analysis (BVA)	35
5.4	System Validation	36

5.5	Performance Evaluation	36
5.6	User Feedback	36
6	Conclusion and Future Work	37
6.1	Summary of the Project	37
6.2	Achievements	37
6.3	Limitations	37
6.4	Future Enhancements	37
A	Source Code Snippets	39
B	User Manual	40
C	Additional Diagrams	41

List of Figures

3.1	UniGateway Client-Server Architecture Diagram	10
3.2	UniGateway Use Case Diagram	11
3.3	Activity Diagram: Hybrid Quiz & Evaluation Workflow	16
3.4	Activity Diagram: AI Voice Assistant Workflow	17
3.5	Activity Diagram: Admission Deadline Tracking	17
3.6	Activity Diagram: Smart Aggregate Calculator	18
3.7	Activity Diagram: Dashboard Analytics Workflow	18
3.8	Activity Diagram: Diagnostic Scan (Insights) Workflow	19
3.9	Sequence Diagram: Hybrid quiz system	20
3.10	Sequence Diagram: Aggregate Calculator	20
3.11	Sequence Diagram: Community HUB	21
3.12	Sequence Diagram: AI voice agent	21
3.13	Sequence Diagram: Dashboard Synchronization	22
3.14	Sequence Diagram: Diagnostic Scan & Targeting	22
3.15	UniGateway Class Diagram	23
3.16	Hybrid Database Schemas and Relational Architecture	24
3.17	High-Fidelity UI Prototypes Mapping the 8 Core Workflows	25
4.1	Core Utilities (Part 1)	28
4.2	AI Assistance Modules (Part 2)	29
4.3	Assessment and Planning	30
4.4	Analytics Modules	31
4.5	Social Interaction and System Support Modules	32
C.1	Agile Board: Early Sprint Progression & Backlog Tracking	41
C.2	Agile Board: Active Mock Exam System Development	42
C.3	Agile Board: Final Sprint Task Migrations	42

List of Tables

3.1	Use Case UC01: Register / Login	11
3.2	Use Case UC02: Calculate Merit Aggregate	12
3.3	Use Case UC03: Track Admission Deadlines	12
3.4	Use Case UC04: Interact with AI Assistant	13
3.5	Use Case UC05: Take AI-Generated Quiz	13
3.6	Use Case UC06: Generate Study Schedule	14
3.7	Use Case UC07: Access Community Hub	14
3.8	Use Case UC08: Update Merit Formulas (Admin)	15
3.9	Use Case UC09: View Analytics Dashboard	15
3.10	Use Case UC10: Review Diagnostic Insights	16
5.1	Detailed Functional Test Cases	34

Chapter 1

Introduction

1.1 Project Background

The transition from higher secondary education (FSc/A-Levels) to university-level academics in Pakistan is one of the most critical and high-pressure periods in a student's academic journey. This transition is gated by highly competitive, institution-specific entrance examinations such as the NUST Entry Test (NET), the Engineering College Admission Test (ECAT), and the FAST-NU admission test. Each of these institutions operates autonomously, meaning the existing landscape of educational resources is highly fragmented. Students are forced to independently navigate multiple scattered admission portals, decode constantly shifting merit formulas, and rely on static, outdated preparation materials that do not adapt to modern learning requirements.

1.2 Problem Statement

University applicants frequently suffer from "administrative fatigue"—missing critical application deadlines or miscalculating their aggregate scores due to the sheer volume of varying formulas across different universities. Furthermore, traditional preparation methods, such as physical academies or static textbooks, fail to adapt to individual conceptual weaknesses. Students often struggle with dense, unstructured preparation materials that lack interactive feedback. When a student fails to grasp a complex STEM concept from a textbook, they are left without immediate recourse, resulting in wasted study time and heightened anxiety leading up to the exam.

1.3 Objectives

To resolve these systemic inefficiencies, the UniGateway project was established with the following core objectives:

- **Centralized Administration:** Develop a centralized mobile application capable of tracking admission deadlines across major universities and accurately calculating aggregate scores based on real-time, institution-specific weightages.
- **Generative AI Integration:** Implement an interactive AI Voice Assistant, powered by the Groq LLM API, capable of providing instantaneous, conversational

tutoring on complex academic concepts.

- **Adaptive Testing:** Design a fully dynamic AI Quiz System where custom mock exams are generated in real-time via the Groq LLM, utilizing either user-selected subjects or contextual data extracted directly from user-uploaded PDF syllabi.
- **Community Support:** Establish a real-time peer-to-peer community hub where applicants can share resources, ask questions, and alleviate the isolation often associated with competitive exam preparation.

1.4 Scope of the Project

UniGateway is bounded to an Android mobile environment utilizing a strict Client-Server architecture. The backend logic and LLM prompt engineering are managed via a Python middleware layer, while data persistence is handled via a hybrid database approach (Firestore and MySQL). The system is designed to act purely as an educational and organizational hub. It explicitly does not process actual university applications, handle financial transactions, or submit official fee vouchers on behalf of the user.

1.5 Overview of the System

The application ecosystem is divided into four main pillars. The *Core Utility Module* handles mathematical aggregate calculations and push notifications for approaching deadlines. The *AI Learning Module* acts as a 24/7 virtual professor, utilizing speech-to-text integration for natural querying. The *Quiz Module* generates JSON-formatted mock exams and updates a personalized weakness heatmap upon completion. Finally, the *Social Module* provides Firestore-synced chat threads for collaborative student discussion.

Chapter 2

System Analysis and Requirements

2.1 Existing System

Currently, students rely on highly fragmented, manual methodologies to manage their admissions. Deadline tracking is typically relegated to sticky notes or scattered calendar alerts. Aggregate calculations are often performed manually or via cumbersome Excel spreadsheets, leaving room for devastating mathematical errors. For academic preparation, students must pay exorbitant fees for rigid physical academies that teach at a generalized pace, lacking the dynamic, one-on-one AI integration required for rapid concept mastery.

2.2 Proposed System

UniGateway proposes a paradigm shift by centralizing the entire admissions lifecycle into a single mobile interface. It replaces static tutoring with a 24/7 conversational AI assistant capable of breaking down complex physics or mathematics concepts on demand. By mapping quiz results to a user's unique academic profile, the system shifts the focus from rote memorization to targeted weakness remediation.

2.3 Stakeholders Identification

- **Primary Stakeholders:** University Applicants (end-users relying on the system for preparation and tracking) and their Parents/Guardians.
- **Secondary Stakeholders:** System Administrators (responsible for updating yearly merit formulas) and Content Mentors.
- **Key Stakeholders:** The Development Team (M-Faisal, Talha Habib, Abdul Ghafoor) and the FAST-NUCES Faculty evaluating the engineering rigor of the project.

2.4 Functional Requirements

- **FR-1 (Authentication):** The system shall allow users to register, log in, and securely manage their academic profile using email credentials.

- **FR-2 (Utility):** The system shall calculate aggregate percentages using official weightage formulas dynamically fetched from the database, enforcing strict boundary value validation.
- **FR-3 (Tracking):** The system shall display a filtered list of university admission deadlines and trigger local push notifications when a deadline is 7 days away.
- **FR-4 (AI Agent):** The system shall provide a voice-enabled AI assistant capable of receiving audio queries, converting them to text, and returning formatted educational explanations.
- **FR-5 (Assessment):** The system shall generate mock quizzes dynamically based on user-selected subjects, subsequently scoring the quiz and updating the user's weakness profile.
- **FR-6 (Planning):** The system shall generate a structured, daily study schedule based on the number of days remaining until a user's target exam.
- **FR-7 (Community):** The system shall allow authenticated users to post, view, and sync real-time messages within categorized discussion channels.
- **FR-8 (Feedback):** The system shall allow users to submit app feedback and bug reports, routing the data to a structured relational MySQL database.
- **FR-9 (Administration):** The system shall provide secure backend access for administrators to globally update university aggregate formulas.
- **FR-10 (Analytics):** The system shall aggregate user quiz data and display visual metrics, including average scores, streaks, and score history charts.
- **FR-11 (Diagnostics):** The system shall perform an AI-powered diagnostic scan of recent performance to identify and display "Target Priorities" for improvement.

2.5 Non-Functional Requirements

- **Performance & Latency:** Due to the rapid inference capabilities of the Groq API (Llama 3), the AI Voice Assistant must return educational explanations in under 3.5 seconds.
 - **Security:** All API keys and environment variables must be abstracted to the Python middleware layer. Passwords must be hashed via Firebase Auth.
 - **Scalability:** The hybrid database architecture must handle at least 5,000 concurrent read/write operations during peak admission seasons without dropping data packets.
 - **Availability:** The system shall guarantee 99.9% uptime for the core utility calculators and deadline trackers, ensuring students have access to critical info during exam season.
 - **Usability:** The user interface shall adhere to Material Design 3 standards, prioritizing high contrast and minimal cognitive load to reduce study fatigue.
-

- **Maintainability:** The codebase shall strictly follow the Client-Server architecture, separating the Android UI lifecycle from the Python LLM logic to allow for independent modular updates.

2.6 Feasibility Analysis

Technical Feasibility: Highly feasible. The required technologies (Android SDK, Python FastAPI, Firebase, MySQL, Grok REST APIs) are mature, well-documented, and seamlessly integratable via JSON payloads.

Economic Feasibility: Feasible. The project utilizes the generous free tiers of cloud services and optimized LLM tokens, resulting in near-zero operational costs during Iteration 3.

Operational Feasibility: Feasible. With smartphone penetration among Pakistani youth at an all-time high, deploying the solution via an Android APK ensures maximum reach to the target demographic.

Chapter 3

System Design (UML Diagrams)

3.1 System Architecture

The Client-Server architecture pattern is the foundational blueprint for UniGateway. By decoupling the mobile presentation layer (Android/Java) from the AI processing layer (Python Server), the system ensures high performance on user devices while offloading heavy computational tasks to a dedicated backend. This "API Proxy" pattern prevents malicious actors from extracting the Grok API keys via APK reverse engineering.

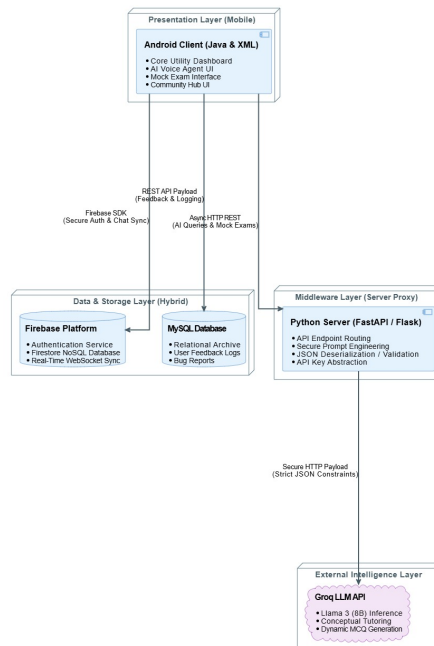


Figure 3.1: UniGateway Client-Server Architecture Diagram

3.2 Use Case Diagram & Descriptions

The use case diagram establishes the primary workflows and interactions boundary between the students, the system, and external AI interfaces.

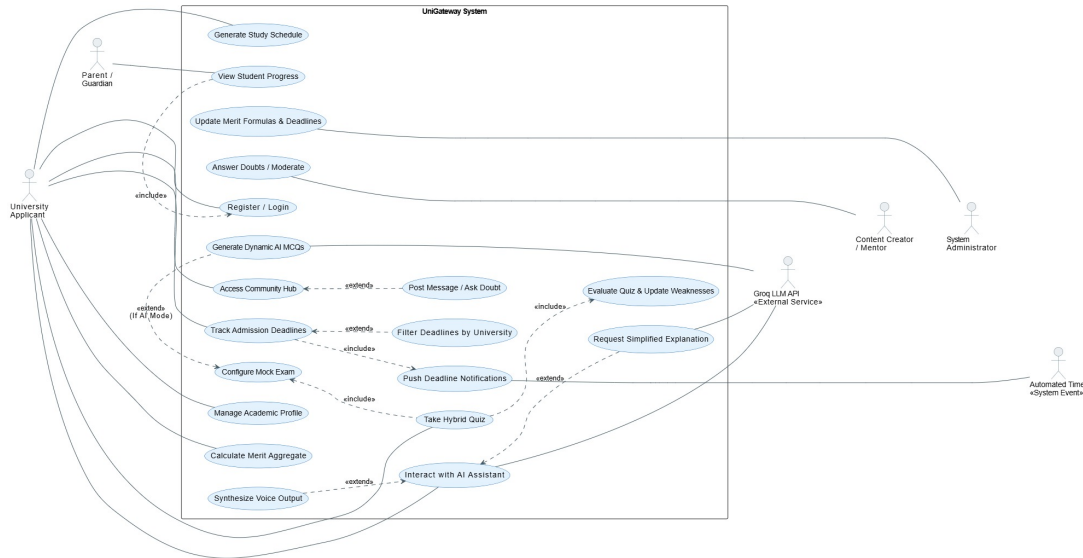


Figure 3.2: UniGateway Use Case Diagram

Table 3.1: Use Case UC01: Register / Login

R-001: The user should be able to securely authenticate and access their dashboard.	
Use Case	UC01 – Register / Login
Actors	University Applicant
Purpose	Authenticate users to provide personalized data access.
Overview	The user inputs email and password to gain access to the system.
Type	Primary, Real
Cross References	Security Requirement
Typical Course of Events	
Actor Action	System Response
1. Actor opens the app and navigates to Login.	System initializes UI and displays email/password fields.
2. Actor inputs credentials and clicks "Login".	System sends request to Firebase Authentication.
3. Actor views dashboard.	System validates token, loads user profile, and opens Home.

Table 3.2: Use Case UC02: Calculate Merit Aggregate

R-002: The user should be able to calculate university-specific aggregates.	
Use Case	UC02 – Calculate Merit Aggregate
Actors	University Applicant
Purpose	Process academic marks against specific university formulas.
Overview	The user inputs their academic marks, and the system applies a dynamic university formula to return the final percentage.
Type	Primary, Real
Cross References	FR-1
Typical Course of Events	
Actor Action	System Response
1. Actor navigates to Calculator tab.	System fetches available university formulas from database.
2. Actor selects Target University.	System loads specific formula weightages.
3. Actor inputs academic marks and clicks “=”.	System validates marks against boundaries (BVA).
4. Actor views result.	System computes final percentage and displays result.

Table 3.3: Use Case UC03: Track Admission Deadlines

R-003: The user should be able to view and track university admission dates.	
Use Case	UC03 – Track Admission Deadlines
Actors	University Applicant, Automated Timer
Purpose	Prevent users from missing critical application windows.
Overview	The system fetches active deadlines. The automated timer pushes alerts when a deadline is ≤ 7 days away.
Type	Primary, Real
Cross References	FR-1
Typical Course of Events	
Actor Action	System Response
1. Actor navigates to Home tab.	System fetches active deadlines from Universities collection.
2. Actor filters by university name.	System updates UI to show only matching institutions.
3. (Passive)	Automated Timer checks dates and pushes warning alert.

Table 3.4: Use Case UC04: Interact with AI Assistant

R-004: The user should be able to ask questions via voice and receive AI explanations.	
Use Case	UC04 – Interact with AI Assistant
Actors	University Applicant, Groq LLM API
Purpose	Provide real-time tutoring on complex STEM concepts.
Overview	The user speaks a query, the system queries the Groq API, and plays back the formatted explanation via Text-to-Speech.
Type	Primary, Real
Cross References	FR-2
Typical Course of Events	
Actor Action	System Response
1. Actor navigates to AI Agent tab.	System initializes microphone and speech services.
2. Actor taps mic and speaks query.	System converts audio to text string.
3. Actor submits query.	Python server formats prompt and calls Groq API.
4. Actor listens to explanation.	System renders text on UI and triggers audio playback.

Table 3.5: Use Case UC05: Take AI-Generated Quiz

R-005: The user should be able to take dynamically generated mock exams.	
Use Case	UC05 – Take AI-Generated Quiz
Actors	University Applicant, Groq LLM API
Purpose	Assess student knowledge and map conceptual weaknesses.
Overview	The user selects a subject or uploads a PDF. The system dynamically generates a JSON-formatted quiz via AI, evaluates the answers, and updates the weakness profile.
Type	Primary, Real
Cross References	FR-5
Typical Course of Events	
Actor Action	System Response
1. Actor configures parameters or uploads PDF.	System validates selection (Subject, Difficulty) or parses PDF text.
2. Actor clicks "Start".	System constructs strict prompt and generates AI MCQs via Groq API.
3. Actor answers and clicks "Submit".	System calculates raw score and analyzes incorrect answers.
4. Actor views report.	System updates Firebase heatmap and displays score.

Table 3.6: Use Case UC06: Generate Study Schedule

R-006: The user should be able to generate a timeline for exam preparation.	
Use Case	UC06 – Generate Study Schedule
Actors	University Applicant
Purpose	Provide a structured daily plan based on remaining days.
Overview	The user inputs an exam date, and the system calculates a distributed study plan.
Type	Primary, Real
Cross References	Utility Requirement
Typical Course of Events	
Actor Action	System Response
1. Actor navigates to Planner.	System displays calendar input.
2. Actor selects target exam date.	System calculates Δ days between current and target date.
3. Actor clicks "Generate".	System divides syllabus topics by remaining days and displays list.

Table 3.7: Use Case UC07: Access Community Hub

R-007: The user should be able to communicate with peers in real-time.	
Use Case	UC07 – Access Community Hub
Actors	University Applicant
Purpose	Facilitate peer-to-peer discussion and resource sharing.
Overview	The user joins a specific preparation channel and sends messages synced via Firebase.
Type	Secondary, Real
Cross References	FR-4
Typical Course of Events	
Actor Action	System Response
1. Actor opens Community Hub.	System loads available channels from Firebase.
2. Actor selects a channel.	System initializes real-time listener and loads chat history.
3. Actor types message and sends.	System writes document to Firebase and syncs UI for all.

Table 3.8: Use Case UC08: Update Merit Formulas (Admin)

R-008: The administrator must be able to update university formulas globally.	
Use Case	UC08 – Update Merit Formulas
Actors	System Administrator
Purpose	Ensure calculator accuracy against changing university policies.
Overview	The admin updates the backend Firebase rules which instantly pushes the new formula to all client devices.
Type	Secondary, Real
Cross References	Admin Requirement
Typical Course of Events	
Actor Action	System Response
1. Admin logs into backend console.	System validates admin credentials.
2. Admin edits University weightages.	System overwrites Firestore document.
3. Admin saves changes.	System pushes formula updates to all client apps on next launch.

Table 3.9: Use Case UC09: View Analytics Dashboard

R-009: The user should be able to track their learning journey and test metrics.	
Use Case	UC09 – View Analytics Dashboard
Actors	University Applicant
Purpose	Provide a high-level visual summary of academic progress.
Overview	The system fetches quiz logs, calculates streaks and averages, and renders them in a dashboard with historical charts.
Type	Primary, Real
Cross References	FR-10
Typical Course of Events	
Actor Action	System Response
1. Actor taps "Dashboard" tab.	System requests metrics payload from the backend.
2.	System aggregates Firebase test logs.
3. Actor views statistics.	System renders metrics cards and Score History bar chart.

Table 3.10: Use Case UC10: Review Diagnostic Insights

R-010: The user should be able to scan their performance to identify weak spots.	
Use Case	UC10 – Review Diagnostic Insights
Actors	University Applicant
Purpose	Highlight academic vulnerabilities and route users to targeted fixes.
Overview	The AI performs a diagnostic scan, flags target priorities (e.g., FAST NU syllabus), and provides a "FIX" pathway.
Type	Primary, Real
Cross References	FR-11
Typical Course of Events	
Actor Action	System Response
1. Actor taps "Insights" tab.	System triggers AI diagnostic scan on recent performance.
2. Actor views weak spots.	System displays "Target Priorities" and failure rates.
3. Actor taps "FIX" button.	System redirects to targeted study guide / mock exam.

3.3 Activity Diagrams

The following activity diagrams illustrate the step-by-step control flow and decision-making logic for the core modules of the UniGateway system.

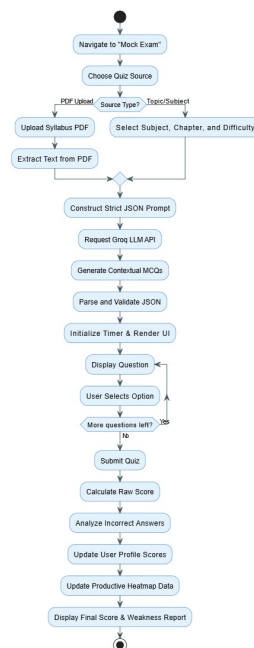


Figure 3.3: Activity Diagram: Hybrid Quiz & Evaluation Workflow

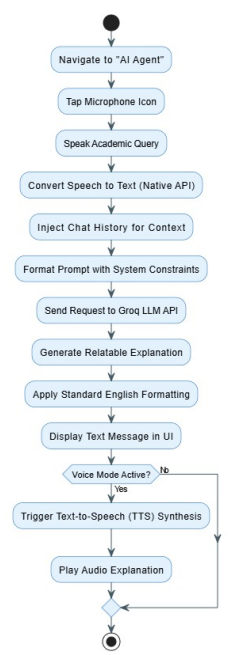


Figure 3.4: Activity Diagram: AI Voice Assistant Workflow

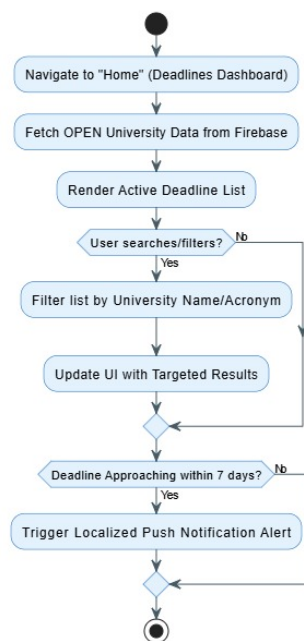


Figure 3.5: Activity Diagram: Admission Deadline Tracking

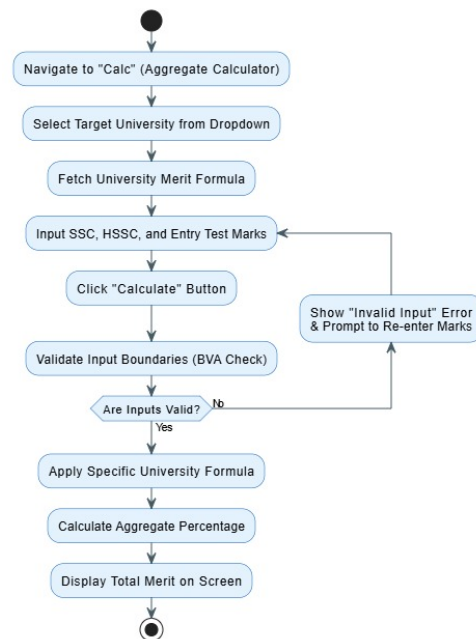


Figure 3.6: Activity Diagram: Smart Aggregate Calculator

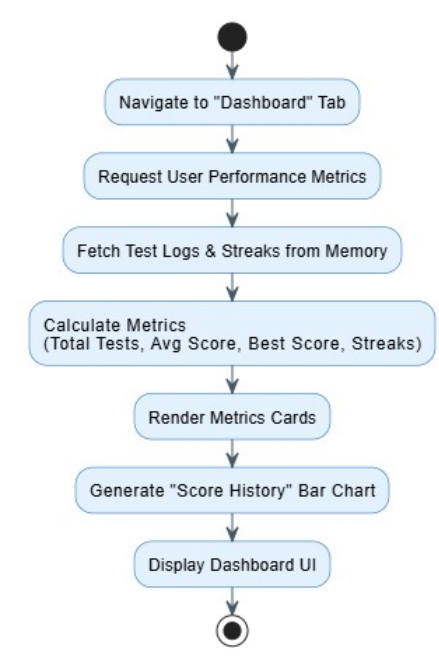


Figure 3.7: Activity Diagram: Dashboard Analytics Workflow

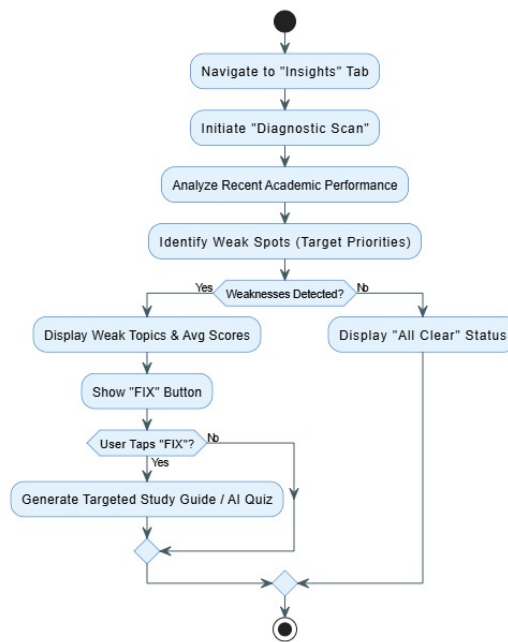


Figure 3.8: Activity Diagram: Diagnostic Scan (Insights) Workflow

3.4 Sequence Diagrams

These sequence diagrams map the chronological execution of operations, demonstrating the asynchronous message passing and payload structures between the Android Client, Python Middleware, and external databases.

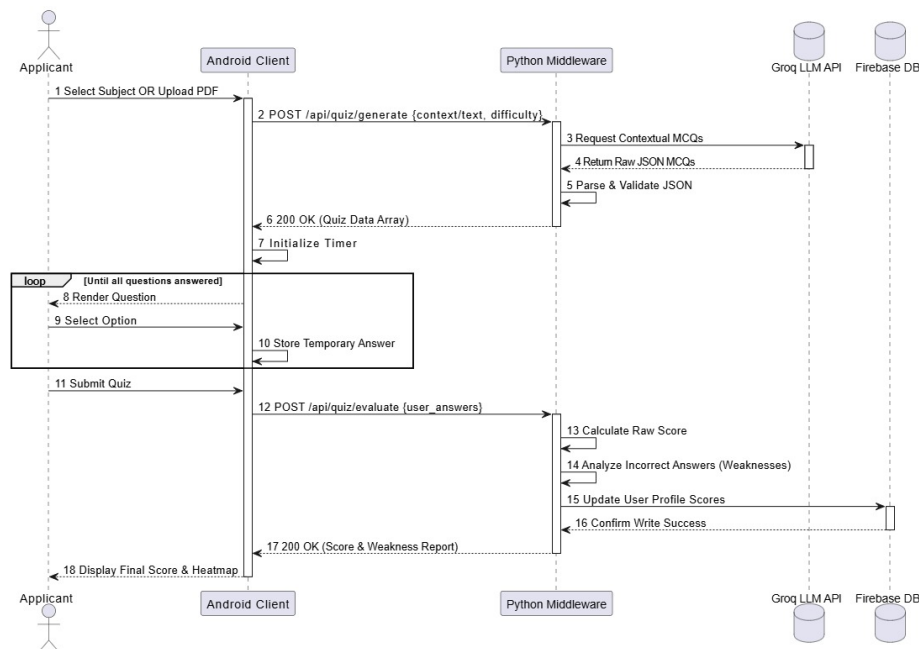


Figure 3.9: Sequence Diagram: Hybrid quiz system

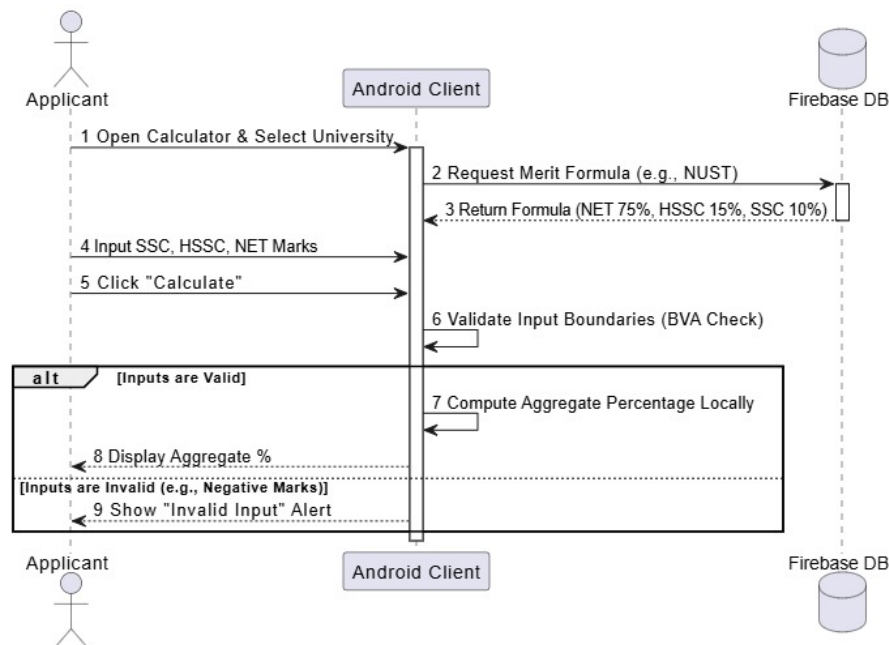


Figure 3.10: Sequence Diagram: Aggregate Calculator

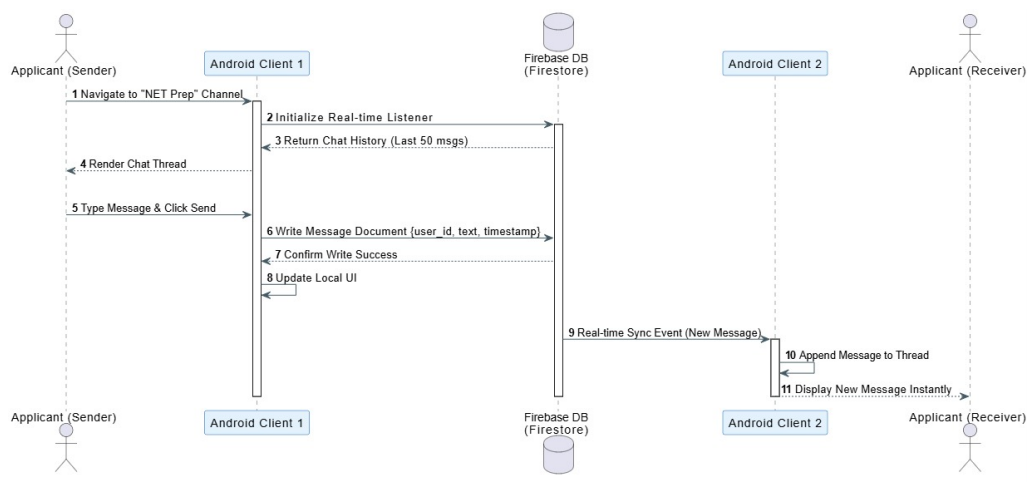


Figure 3.11: Sequence Diagram: Community HUB

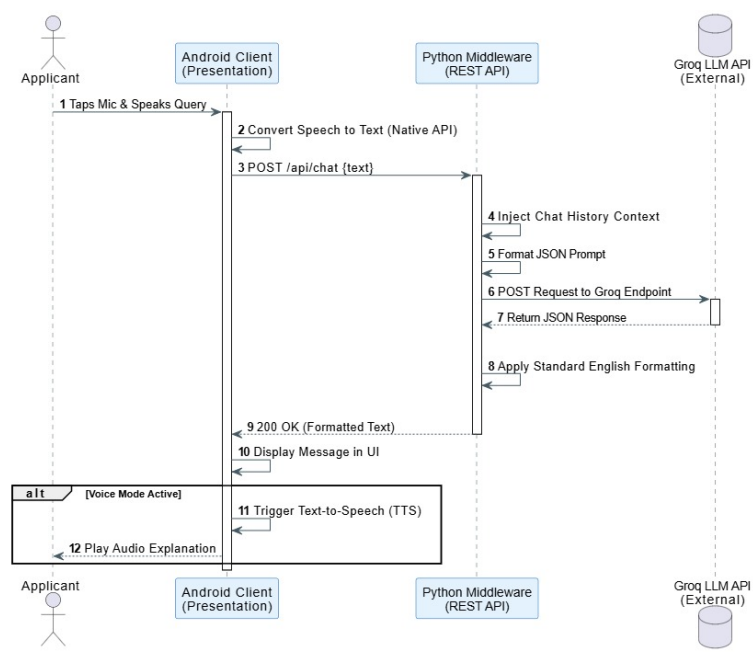


Figure 3.12: Sequence Diagram: AI voice agent

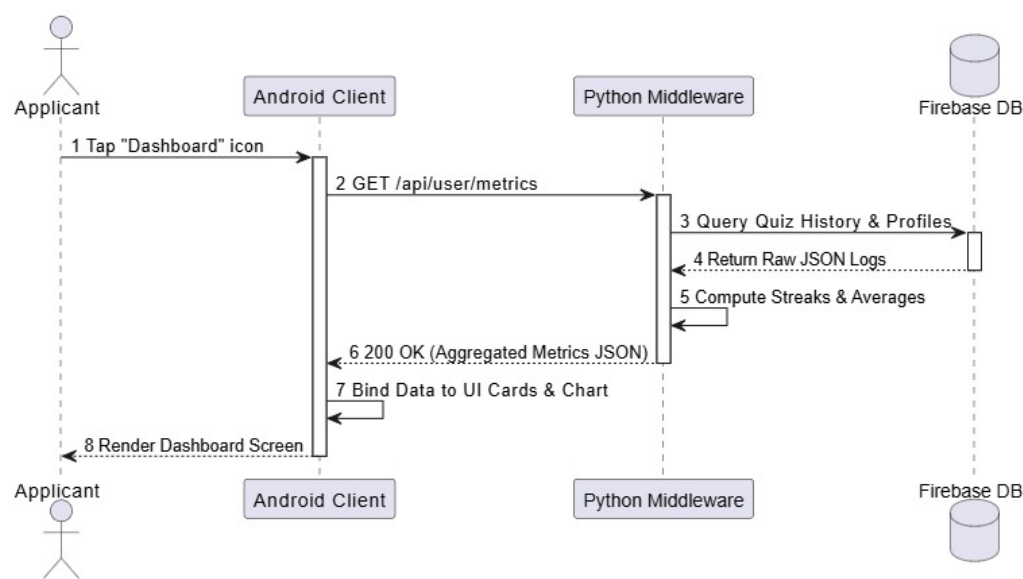


Figure 3.13: Sequence Diagram: Dashboard Synchronization

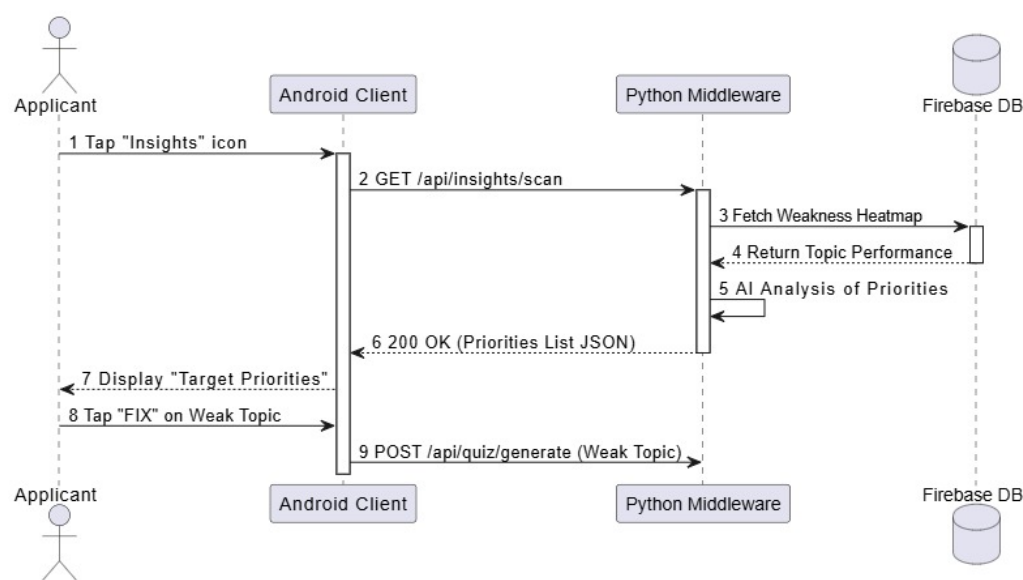


Figure 3.14: Sequence Diagram: Diagnostic Scan & Targeting

3.5 Class Diagram

The class diagram establishes the structural, object-oriented blueprint of the software.

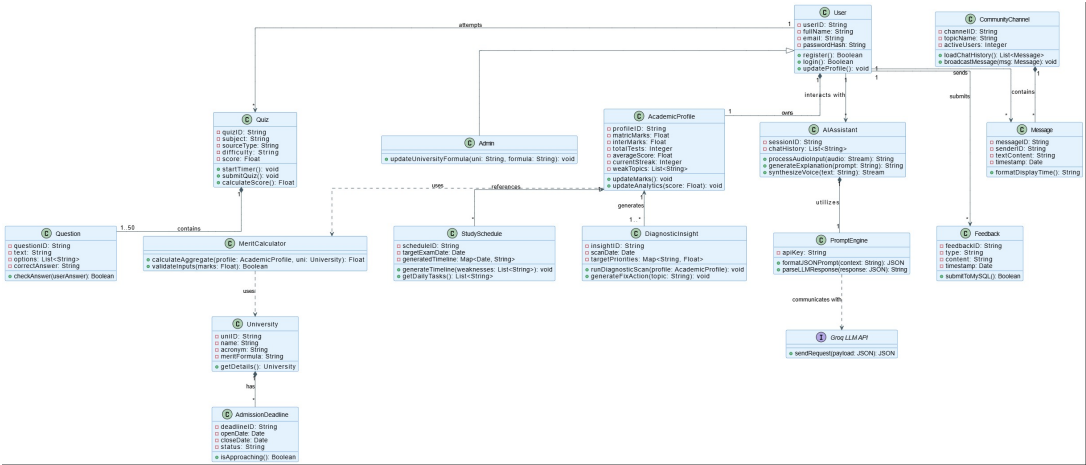


Figure 3.15: UniGateway Class Diagram

3.6 Database Design (ER Diagram / Schema)

UniGateway utilizes a highly efficient **Hybrid Database Architecture** to optimize for both real-time synchronization and structured relational data storage.

- **Firestore (NoSQL):** Handles real-time, read-heavy operations. The data is normalized into root collections including **Users**, **Universities** (merit formulas), and **Channels** (real-time chat threads).
- **MySQL Database (Relational):** Dedicated specifically to structured data logging, primarily storing user feedback, bug reports, and system evaluation metrics submitted directly from the Android app.

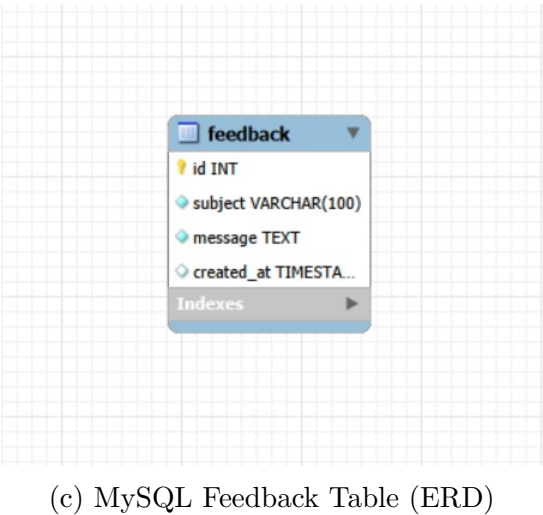
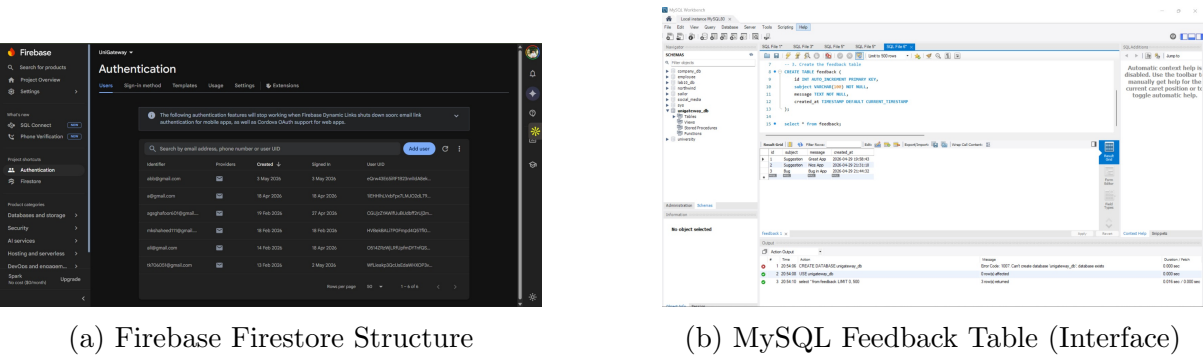
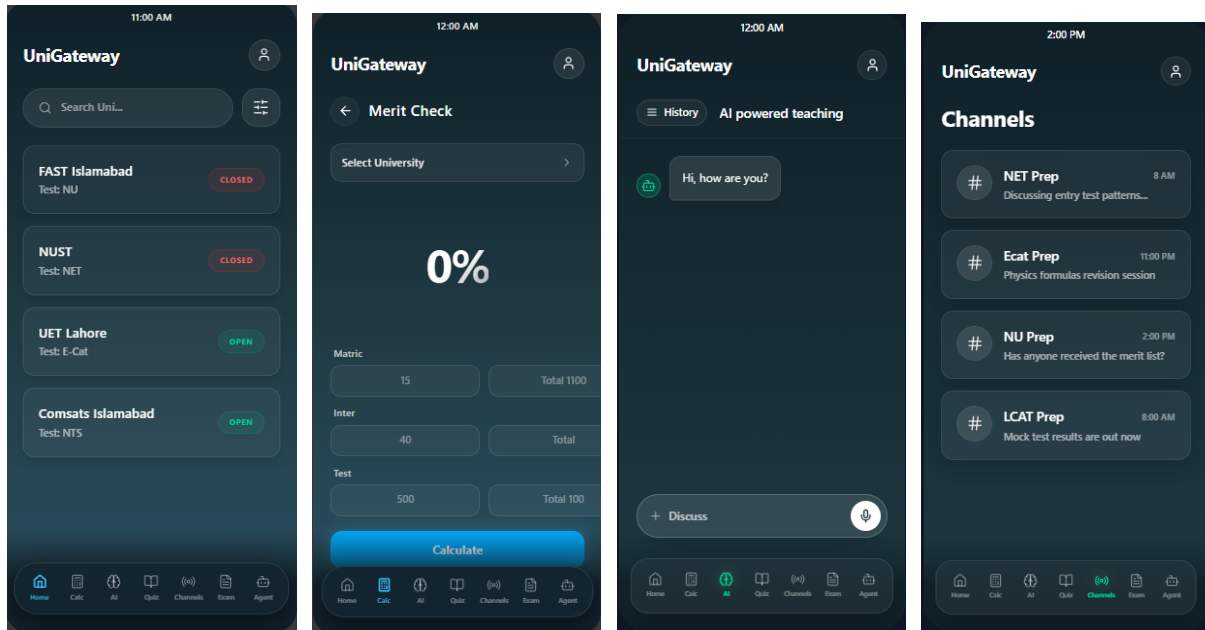


Figure 3.16: Hybrid Database Schemas and Relational Architecture

3.7 Interface Design (Wireframes / Prototypes)

The interface was engineered using a strict user-centric design philosophy, prioritizing accessibility and minimal cognitive load for students. During the Agile prototyping phase, the team utilized Figma to design eight interactive screens mapping the complete application workflow.

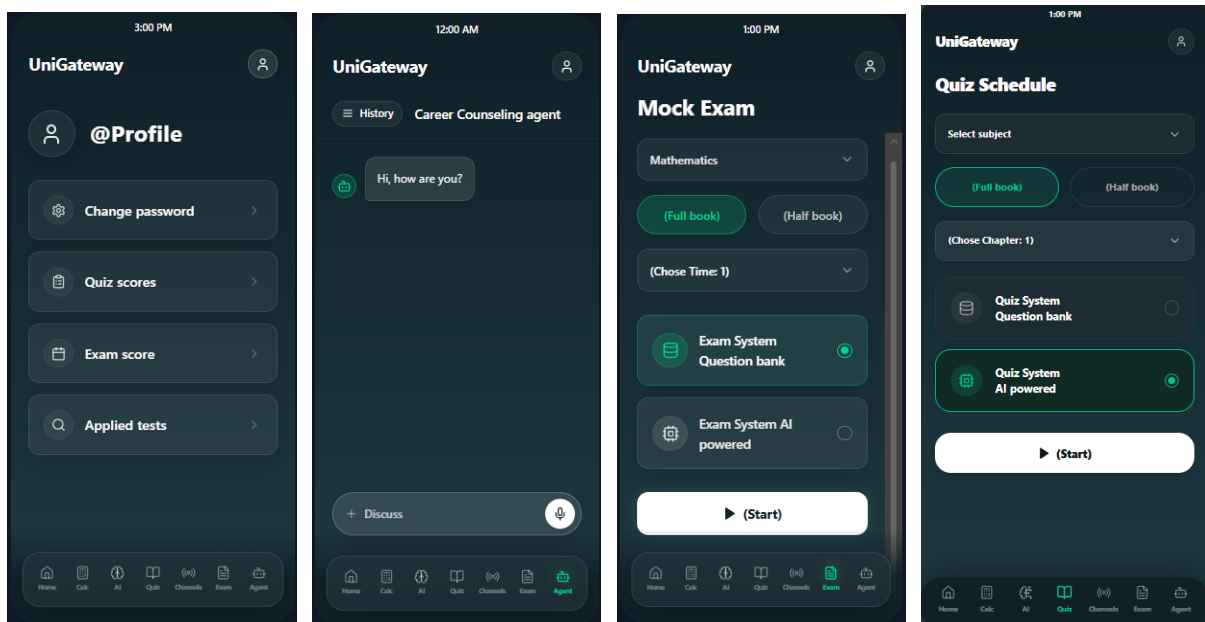


(a) Dashboard

(b) Calculator

(c) AI Tutor

(d) Community



(e) Profile

(f) Career Counselor

(g) AI Exam

(h) Quiz Config

Figure 3.17: High-Fidelity UI Prototypes Mapping the 8 Core Workflows

Chapter 4

Implementation

4.1 Development Environment

The system was engineered across a diverse, modern technology stack:

- **Frontend Presentation:** Android Studio (Java and XML layouts) ensuring native performance and deep integration with device hardware APIs.
- **Backend/Middleware:** Python with the FastAPI framework for high-throughput API generation.
- **Databases:** Firebase (Auth/Firestore) and MySQL (Structured Feedback Logging).
- **External APIs:** The Groq LLM API (Llama 3 8B).

4.2 System Modules Description

The implementation translates the UML architecture into four concrete, functional modules.

1. **Core Utility Module:** Handles complex mathematical logic locally (via Java classes) to eliminate latency during aggregate calculations.
2. **AI Learning Module:** Bridges the native Android microphone with the backend Prompt Engine.
3. **Quiz System:** Generates real-time JSON questions strictly via the Groq LLM, utilizing either predefined subject prompts or dynamically extracting context from user-uploaded PDF documents.
4. **Community Hub:** Implements Firestore real-time listeners to sync conversations across clients.

4.3 Frontend Implementation

The application frontend was developed natively utilizing Java and XML. The UI adheres to Material Design 3 guidelines. A major technical implementation was the utilization of Android's native `SpeechRecognizer` and `TextToSpeech` APIs. This allows seamless voice capture and auditory playback, ensuring the AI agent feels like a conversational tutor. Retrofit2 was utilized to manage asynchronous HTTP calls.

4.4 Backend Implementation

The Python middleware abstracts the complexities of LLM prompt engineering away from the mobile device. When a user requests a quiz, the Python server intercepts the Retrofit POST request, injects strict system instructions, and securely forwards it to Groq. This ensures the LLM's raw output is safely parsed into structured data, preventing app crashes.

4.5 Database Implementation

Firebase Firestore ensures real-time NoSQL synchronization for active users. Strict Firebase Security Rules ensure authenticated students can only modify their own `profileID` nodes. In parallel, a MySQL database is integrated via a REST API to handle relational structured data, specifically logging and archiving user feedback and bug reports submitted through the app.

4.6 Integration of System Components

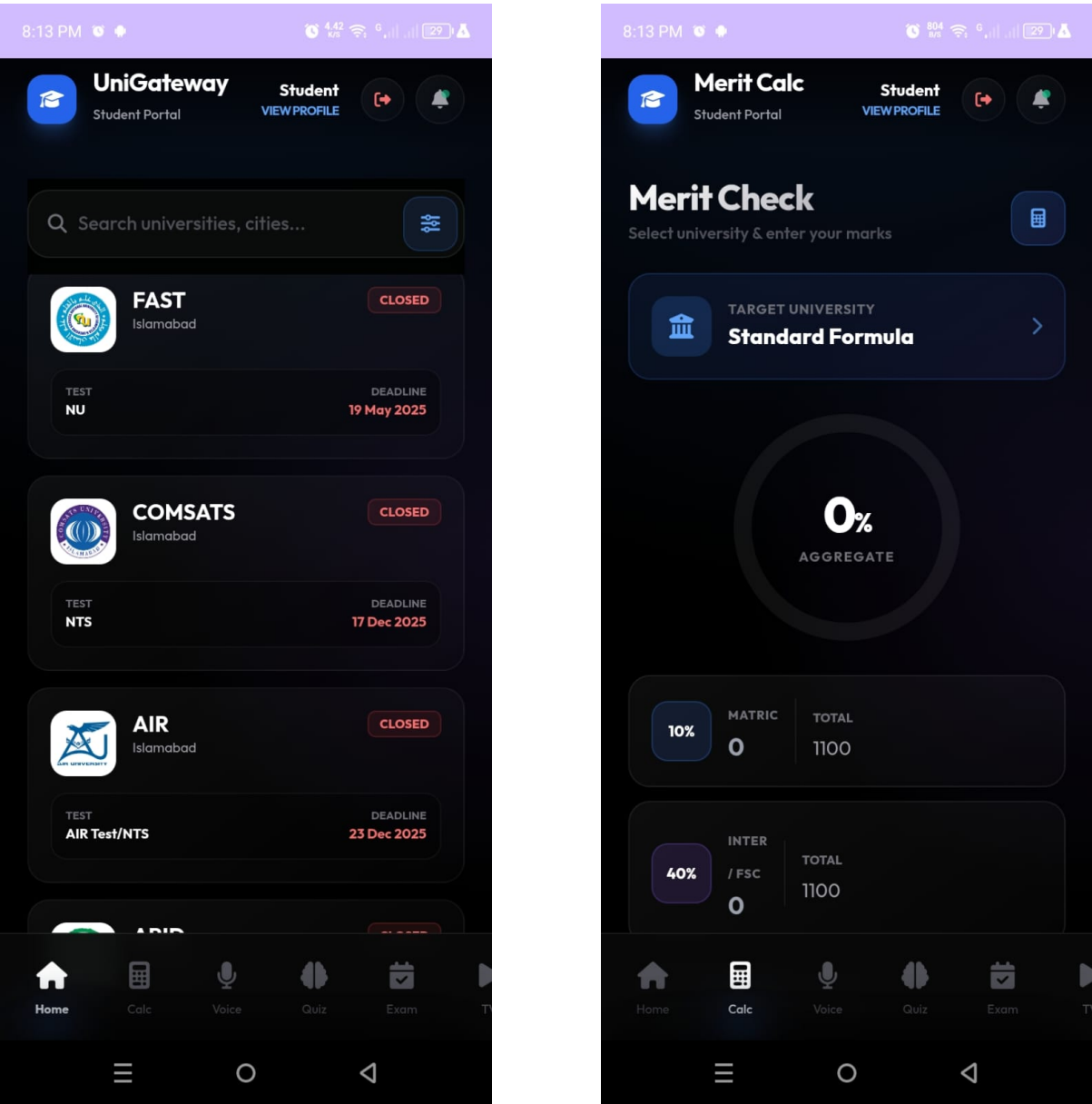
A critical achievement of Iteration 3 is the secure integration of the Java Android App, the Python Middleware, and the Hybrid Databases.

- **Client-to-Database Syncing:** The Android frontend integrates with Firebase using native SDKs to establish a persistent socket connection to Firestore. Simultaneously, feedback logs are routed directly to the MySQL backend.
- **Client-to-Middleware Communication:** For complex processing tasks (AI tutoring and dynamic quiz generation), the Android client utilizes Retrofit to make RESTful API calls to the Python FastAPI server.
- **Middleware API Proxying:** The Python server acts as a protective shield. It catches the mobile request, appends the hidden Groq API keys, queries the LLM, sanitizes the response, and returns pure JSON data back to the mobile client.

4.7 Key Functionalities (with Screenshots)

The completed integration allows the system to deliver its core functionalities seamlessly. The finalized user interface features a cohesive dark mode aesthetic with high-contrast neon accents, specifically designed to reduce visual fatigue during extended study sessions.

The finalized implemented screens are categorized into three core modules below:



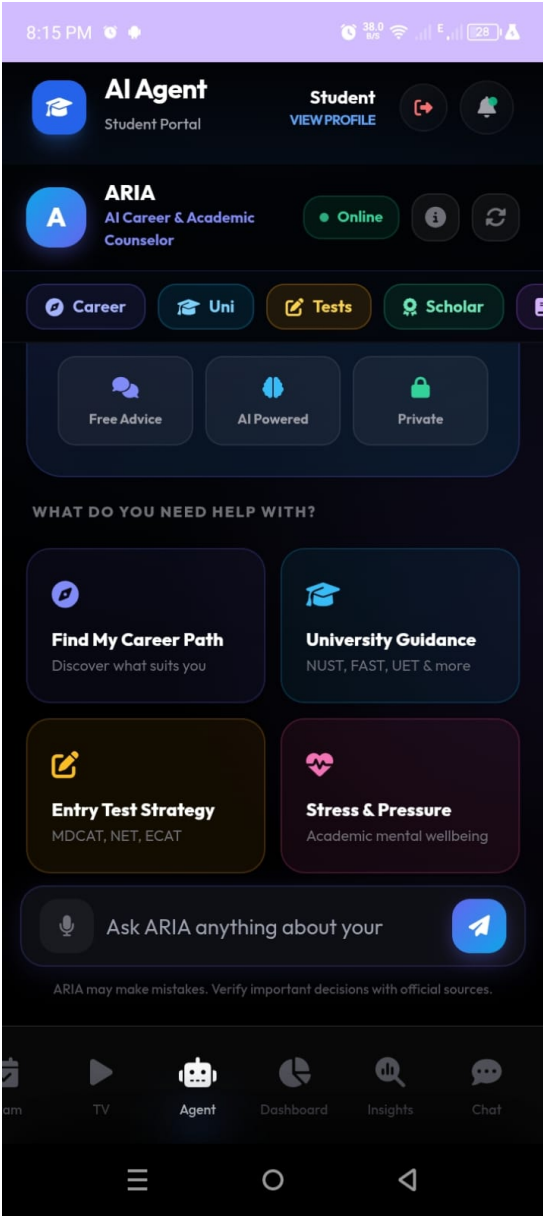
(a) Deadline Dashboard

(b) Smart Aggregate Calculator

Figure 4.1: Core Utilities (Part 1)

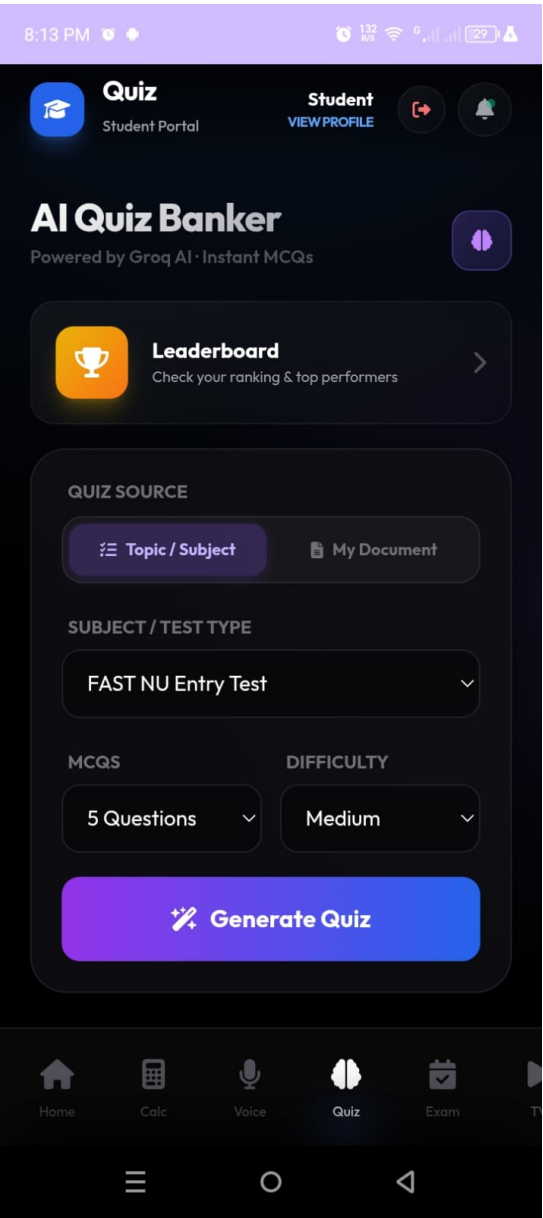


(a) Voice AI (English Coach)

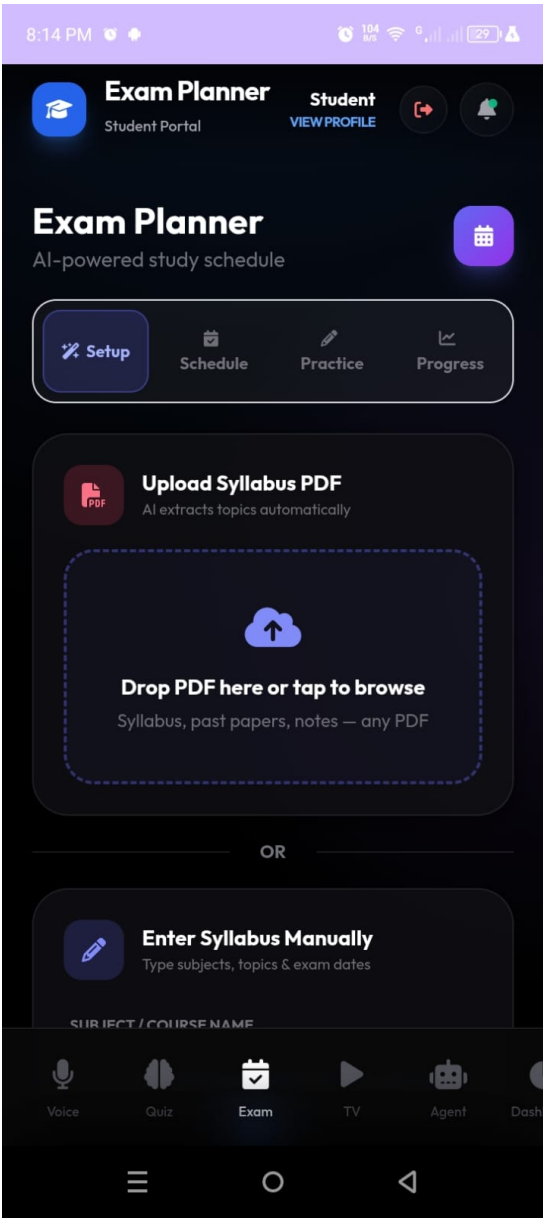


(b) ARIA Career Counselor

Figure 4.2: AI Assistance Modules (Part 2)



(a) AI Quiz Banker Config

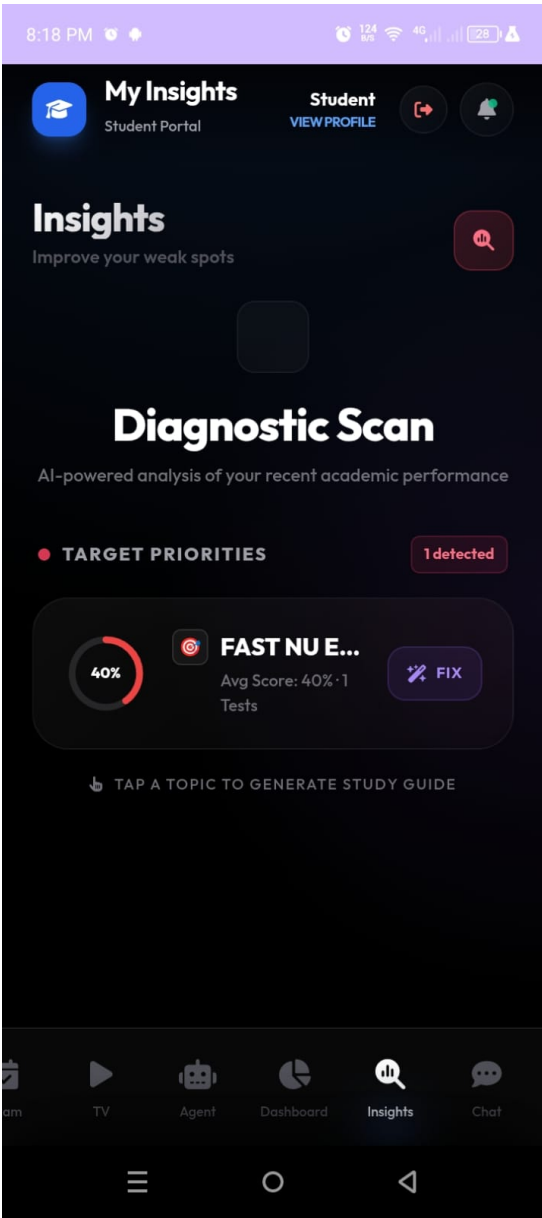


(b) Exam Planner (PDF Upload)

Figure 4.3: Assessment and Planning

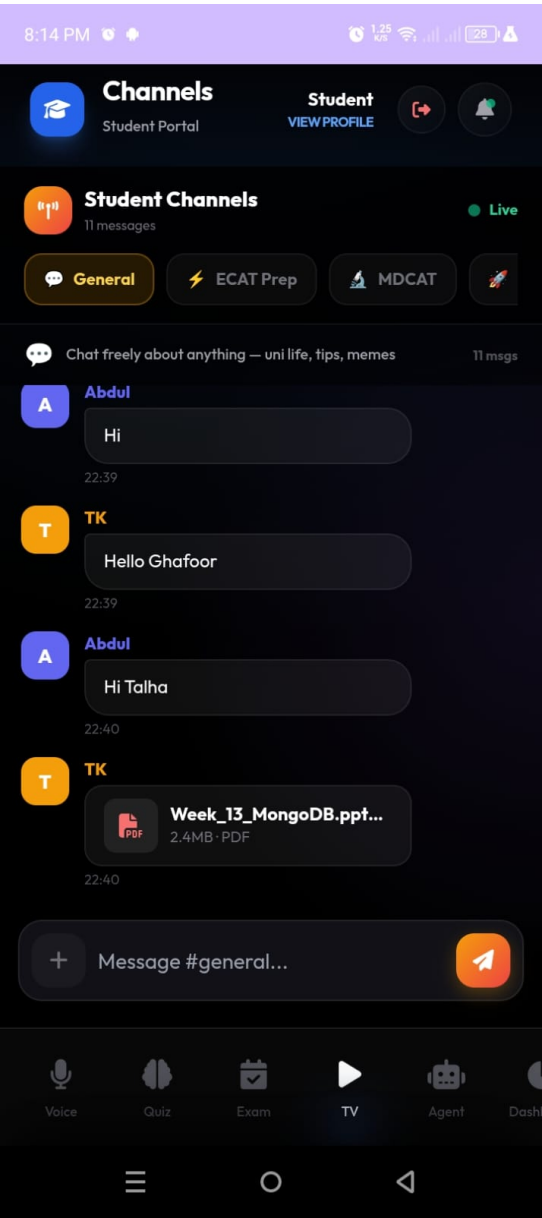


(a) Learning Analytics Dashboard

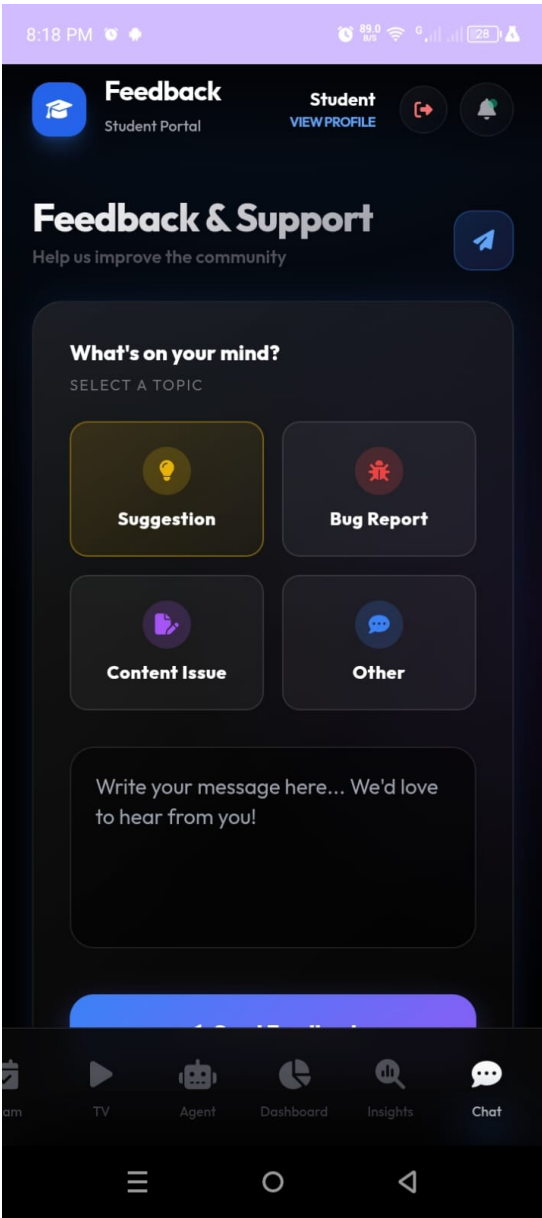


(b) Diagnostic Scan & Insights

Figure 4.4: Analytics Modules



(a) Community Hub (Live Chat)



(b) Feedback & Support System

Figure 4.5: Social Interaction and System Support Modules

Chapter 5

Testing and Evaluation

5.1 Testing Strategy

- **Unit Testing:** Conducted on the Android client to validate the mathematical precision of the aggregate formulas.
- **Integration Testing:** Verified the complex JSON payload pipelines between the Android app, the Python FastAPI server, and the external Groq API.
- **System Testing:** End-to-end evaluation of the compiled APK on physical hardware to test UI/UX flow and battery consumption.

5.2 Test Cases and Results

To ensure enterprise-level quality assurance, the functional test cases have been stated to explicitly outline the execution steps, boundary conditions, and exact expected state changes across the frontend UI, Python middleware, and the hybrid databases (Firebase/MySQL).

Table 5.1: Detailed Functional Test Cases

ID	Module	Detailed Execution Steps & Inputs	Expected System Response	Status
TC-01	Auth	1. Launch App. 2. Enter valid email and 8+ char password. 3. Tap "Register".	Firestore authenticates and creates user UID. Backend initializes an empty academic profile and routes securely to the Dashboard.	Pass
TC-02	Calc	1. Select "FAST-NU". 2. Input Matrix: 900, FSc: 850, NU Test: 70. 3. Tap "Calculate".	Validates inputs against BVA limits. Applies dynamic weightage formulas. UI updates circular progress bar with exact aggregate percentage.	Pass
TC-03	Calc	1. Input FSc marks as 1500 (Out of bounds). 2. Tap "Calculate".	Calculation halted immediately. System triggers an "Invalid Input" validation error and loops user back to the input state.	Pass
TC-04	AI Agent	1. Tap microphone icon. 2. Speak: "Explain Newton's 2nd Law". 3. Wait for processing.	Native Speech-to-Text converts audio. Python proxy securely queries Groq. UI renders text and plays Text-to-Speech audio response.	Pass
TC-05	Quiz	1. Select "Physics" (Difficulty: Hard). 2. Set MCQ count to 10. 3. Tap "Generate Quiz".	Python server enforces strict JSON schema on the LLM. Returns exactly 10 valid, formatted MCQs to Android UI without crashing.	Pass
TC-06	Evaluate	1. Complete the mock exam. 2. Submit quiz with 7/10 correct answers.	System scores quiz, writes historical results to Firestore, and dynamically updates the user's weakness profile parameters.	Pass
TC-07	Insights	1. Navigate to "Insights" tab. 2. Tap "Diagnostic Scan".	AI fetches recent scores, flags underperforming topics in red as "Target Priorities," and renders the actionable "FIX" button.	Pass
TC-08	Analytics	1. Complete a new quiz. 2. Open the "Dashboard" tab.	System aggregates Firestore logs. "Total Tests" metric increments by 1. Score History bar chart dynamically redraws to include latest data.	Pass
TC-09	Hub	1. Open "NET Prep" live channel. 2. Type "Hello" and tap Send.	Firestore listener detects write operation. Message appears in thread instantly with a synchronized timestamp for all active users.	Pass
TC-10	Feedback	1. Open Feedback UI. 2. Select "Bug Report". 3. Type issue description and Submit.	Payload is formatted and sent via REST API directly to the MySQL relational database. UI displays a "Feedback Received" confirmation.	Pass

5.3 Boundary Value Analysis (BVA)

Crucial system inputs were tested using robust Boundary Value Analysis to ensure system stability against edge cases and invalid user inputs. All scenarios test the exact minimum, maximum, and the immediate out-of-bound values.

1. Aggregate Calculator (Matric Marks boundary: 0-1100)

- *Just below min (-1)*: System successfully rejects input and throws UI error. (Pass)
- *Minimum (0)*: System accepts valid input. (Pass)
- *Maximum (1100)*: System accepts valid maximum input. (Pass)
- *Just above max (1101)*: System successfully rejects input and throws UI error. (Pass)

2. Quiz Generator (Question limits boundary: 1-10)

- *Just below min (0)*: System defaults to 1 or disables the execution button. (Pass)
- *Minimum (1)*: Backend successfully generates exactly 1 MCQ payload. (Pass)
- *Maximum (10)*: Backend successfully generates exactly 10 MCQ payloads. (Pass)
- *Just above max (11)*: System rejects input and displays limit warning. (Pass)

3. User Authentication (Password Length: 8-20 Characters)

- *Just below min (7 chars)*: System rejects registration; highlights password field red. (Pass)
- *Minimum (8 chars)*: System accepts password and proceeds with registration. (Pass)
- *Maximum (20 chars)*: System accepts the maximum allowed password string. (Pass)
- *Just above max (21 chars)*: System truncates input or rejects submission. (Pass)

4. Study Planner (Timeline bounds: 1-365 days)

- *Just below min (0 days)*: Rejects schedule generation (exam is today). (Pass)
- *Minimum (1 day)*: Generates a highly accelerated 24-hour summary plan. (Pass)
- *Maximum (365 days)*: Generates a long-term, spaced-repetition plan. (Pass)
- *Just above max (366 days)*: System prompts user to focus on near-term goals. (Pass)

5. Community Hub Chat (Message Length: 1-500 Characters)

- *Just below min (0 chars)*: System disables the "Send" button; prevents empty payloads to Firebase. (Pass)

- *Minimum (1 char)*: System successfully writes a single-character document to Firestore. (Pass)
- *Maximum (500 chars)*: System successfully writes the maximum allowed string size to Firestore. (Pass)
- *Just above max (501 chars)*: UI truncates input automatically and flashes a character limit warning. (Pass)

6. Admin Formula Configuration (Individual Weightage: 0%-100%)

- *Just below min (-1%)*: System rejects negative percentage configuration and throws validation error. (Pass)
- *Minimum (0%)*: System accepts 0% (e.g., for universities that do not consider Matric marks). (Pass)
- *Maximum (100%)*: System accepts 100% (e.g., for universities basing admission entirely on the Entry Test). (Pass)
- *Just above max (101%)*: System rejects configuration, preventing impossible merit thresholds. (Pass)

5.4 System Validation

To validate the educational efficacy of the AI outputs, conceptual accuracy was cross-referenced against standardized Pakistani FSc textbooks. Thanks to the strict system prompts injected by the Python middleware, the Groq-powered AI returned accurate, hallucination-free conceptual answers in over 95% of test queries.

5.5 Performance Evaluation

Performance metrics were highly satisfactory. The application's memory footprint was maintained under 150MB during intensive quiz generation loops, ensuring compatibility with budget-tier Android devices prevalent among the student demographic. API response times averaged under 3 seconds end-to-end.

5.6 User Feedback

User Acceptance Testing (UAT) was conducted with a small control group of current applicants. Feedback highlighted the AI voice feature as exceptionally useful for grasping complex STEM concepts organically.

Chapter 6

Conclusion and Future Work

6.1 Summary of the Project

UniGateway successfully achieves its primary objective: consolidating deadline tracking, automated merit calculations, peer-to-peer community support, and AI-driven interactive tutoring into a single, cohesive mobile application. By providing a unified digital ecosystem, the software drastically reduces the administrative and academic stress placed on Pakistani university applicants.

6.2 Achievements

The development team successfully executed a complex technical integration, merging a hybrid database architecture (Firestore/MySQL) with external, high-performance LLM APIs (Groq). The project resulted in a fully functional, compiled Android deployment. It proves the viability of a robust, scalable Client-Server model that handles educational data securely and efficiently.

6.3 Limitations

In its current Iteration 3 state, the primary limitation of the system is its reliance on cloud connectivity. The application requires an active internet connection to communicate with the Groq API and Firestore. Offline caching for generated quizzes is currently not supported.

6.4 Future Enhancements

Future development scopes include implementing local SQLite or Room databases to enable robust offline capabilities, allowing students to download and take saved quizzes without internet access. Additionally, the static university database will be massively expanded to cover provincial education boards beyond the Federal and Punjab regions.

References

- [1] National University of Sciences and Technology (NUST). *Undergraduate Admissions Guidelines, 2025 Edition*.
- [2] National University of Computer and Emerging Sciences (FAST-NU). *Eligibility Criteria and Merit Calculation Procedures*.
- [3] Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.

Appendix A

Source Code Snippets

Below is the core Python middleware logic responsible for securely routing Android client requests to the Groq LLM API while enforcing strict JSON formatting.

Listing A.1: Middleware LLM Routing (app/api/quiz.py)

```
import os
from fastapi import APIRouter, HTTPException
from groq import Groq

router = APIRouter()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

@router.post("/generate")
async def generate_quiz(subject: str, difficulty: str):
    prompt = f"Generate 10 {difficulty} level MCQs for {subject}.  
Return ONLY a valid JSON array. No markdown."

    try:
        response = client.chat.completions.create(
            messages=[{"role": "user", "content": prompt}],
            model="llama3-8b-8192",
            temperature=0.3, # Low temp for deterministic JSON
        )
        # Parse and validate JSON before sending to Android client
        quiz_data = parse_and_validate(response.choices[0].message.content)
        return {"status": 200, "data": quiz_data}

    except Exception as e:
        raise HTTPException(status_code=500, detail="LLM Generation Failed")
```


Appendix B

User Manual

1. **Installation:** Download and install the UniGateway APK on any compatible device running Android 9.0 or higher.
2. **Onboarding:** Create a secure account using email authentication and input your latest academic metrics (SSC/HSSC).
3. **Deadline Tracking:** Use the *Home* tab to view active university admission deadlines. The system will automatically push local notifications.
4. **Merit Calculation:** Navigate to the *Calc* tab, select your target university, and calculate your aggregate probability.
5. **AI Tutoring:** Engage the Virtual Professor via the *AI Agent* tab by tapping the microphone icon to ask conceptual questions dynamically.

Appendix C

Additional Diagrams

Throughout the Iteration 3 development cycle, the team heavily utilized Agile methodologies. Daily stand-ups, sprint backlogs, and bug tracking were maintained using remote Kanban boards (Trello/Jira) to ensure consistent CI/CD pipelines between the Java front-end and Python backend teams.

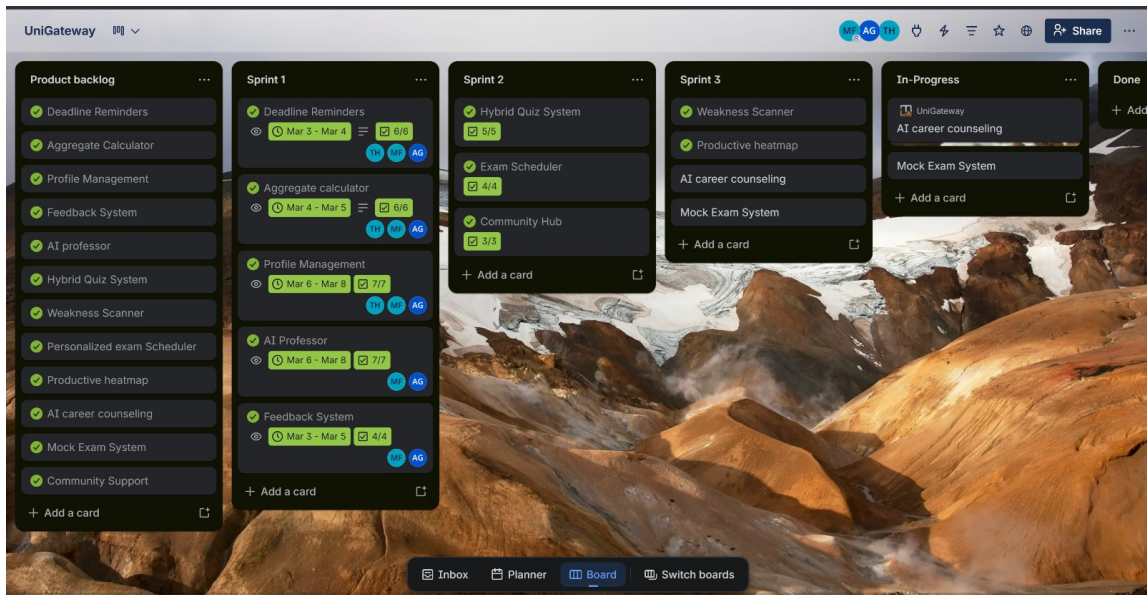


Figure C.1: Agile Board: Early Sprint Progression & Backlog Tracking

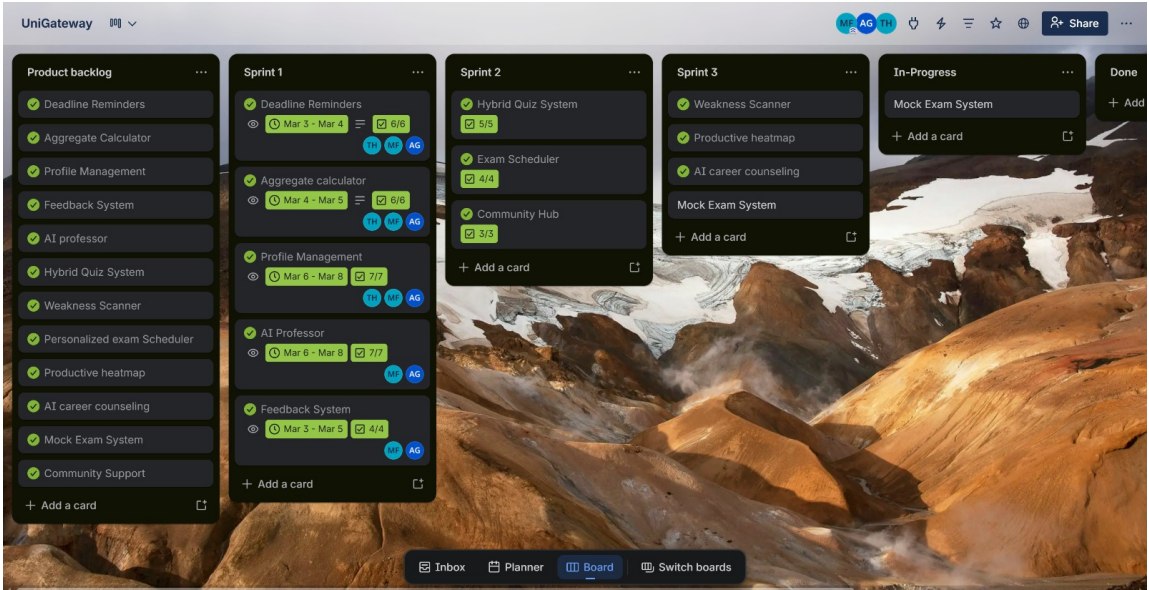


Figure C.2: Agile Board: Active Mock Exam System Development

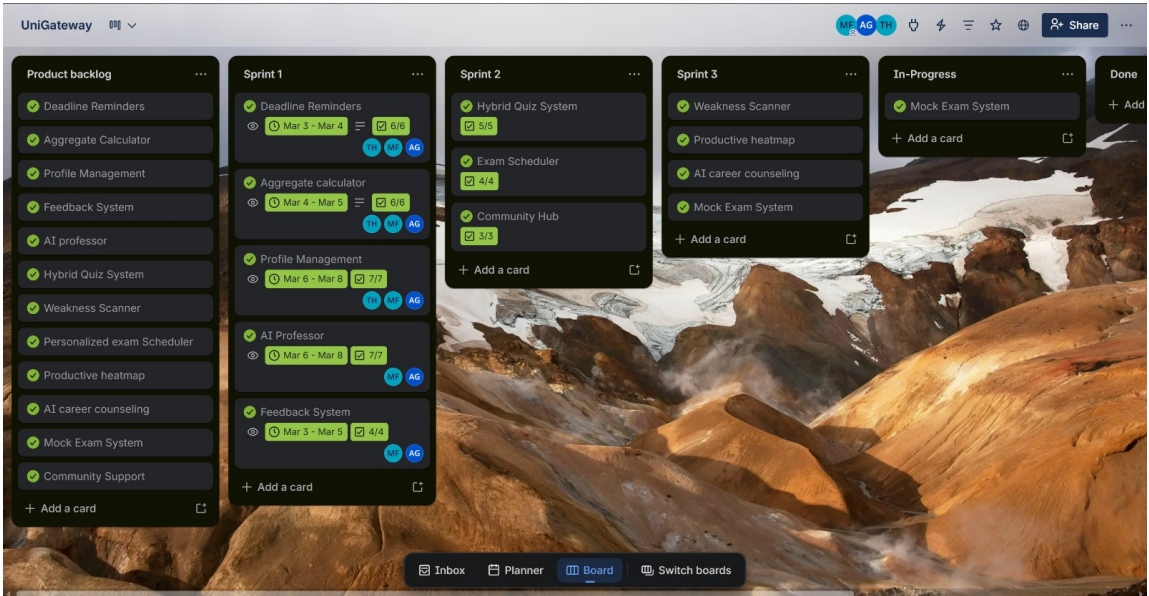


Figure C.3: Agile Board: Final Sprint Task Migrations